MERN STACK

TRAINING TR-103 REPORT DAY 27

25 July 2026

Overview:

Day 27 of the MERN Stack training focused on developing the Login Page for the CareerConnect – Career Management System. I learned how to create a user authentication interface using React.js, manage form inputs using the useState() Hook, and navigate users to different pages using the useNavigate() Hook from React Router. The Login Page was connected with the backend Login API using the Fetch API and a POST request. I also learned how to store authenticated user information such as user role, user name, and user ID in localStorage and redirect users according to their role.

Learning Objectives:

- Understand the development of a Login Page using React.js.
- Learn how to manage login form inputs using `useState()`.
- Understand the use of `useNavigate()` for page navigation.
- Connect the React Login Page with the backend Login API.
- Send login credentials using the Fetch API.
- Handle successful and unsuccessful authentication.
- Learn how to store user information using `localStorage`.
- Understand role-based navigation after login.

Project : CareerConnect - Career Guidance and Management System

LOGIN PAGE DEVELOPMENT

• The Login Page is developed as an authentication module for the CareerConnect career management system.
• The useState() hook is used to manage input values such as email and password and control the login form state.
• The useNavigate() hook from react-router-dom is used to navigate users to different pages after successful authentication.
• The Login Page provides a simple and user-friendly interface for existing users to access the CareerConnect application.
• The Login Page mainly consists of a Login Form for entering registered user credentials.

Login Form

• The Login Form allows existing CareerConnect users to enter their registered email and password.
• User credentials are sent to the backend using the Fetch API.
• The email and password entered by the user are sent to the backend in JSON format.
• The backend verifies the entered credentials with the user information stored in MongoDB.
• After successful authentication, user information received from the server can be stored in `localStorage` for maintaining the user's login session.
• The stored information can include:

1. User Role
2. User Name
3. User ID

Working

1. The `Login()` functional component is created to handle the login functionality of the CareerConnect application.

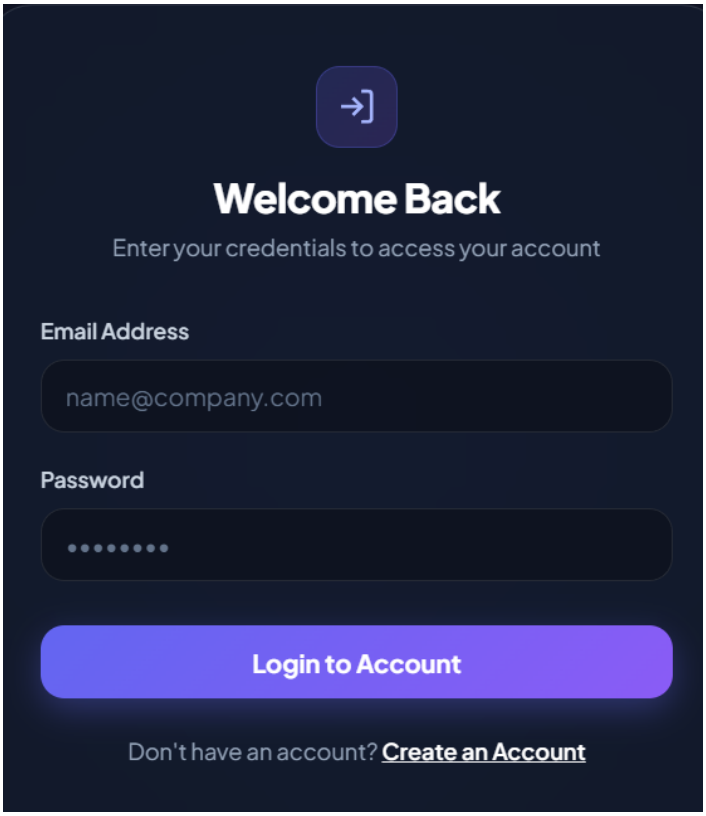
2. The `useState()` hook is used to store and manage the email and password entered by the user.
3. The user enters their registered email and password into the login form.
4. When the Login button is clicked, the `handleLogin()` function is executed.
5. The `handleLogin()` function sends the entered email and password to the backend using a POST request.
6. The backend receives the credentials and searches for the user in the MongoDB database.
7. The server verifies the user's email and password.
8. If authentication is successful, the user details and `usertype` are received from the backend.
9. The application stores the required user information in `localStorage`.
10. Based on the user's role, the application navigates the user to the respective dashboard or page.
11. If authentication fails, the system displays an appropriate error message such as invalid password or user not found.

Purpose

The purpose of the Login Page is to provide secure and convenient access to the CareerConnect career management system. It authenticates registered users, identifies their roles, and provides access to the appropriate features and dashboard based on their user type.

Conclusion:

By the end of Day 27, I gained a clear understanding of developing a Login Page for the CareerConnect – Career Management System. I learned how to manage login form inputs using the useState() Hook, connect the React frontend with the backend Login API using the Fetch API, handle authentication responses, and store user information using localStorage. I also learned how to use useNavigate() for navigation and implement role-based redirection using `usertype`. This strengthened my understanding of React forms, API integration, authentication, local storage, and navigation in MERN Stack applications.


Welcome Back
Enter your credentials to access your account
Email Address
name@company.com
Password
Login to Account
Don't have an account? Create an Account